

SoundScore V1 — Mobile Architecture Report

Author: Madhav Chauhan **Date:** 2026-03-19 **Audit Pass:** 9

Mobile Strategy

Native Android (Kotlin + Jetpack Compose) + Native iOS (SwiftUI). Shared Node.js/Express backend deployed on Railway. No shared mobile code layer (no KMP, no React Native).

Why native:

- Platform-native UX patterns (iOS UINavigationController, Android NavHost)
 - Glassmorphism/frosted-glass effects that require platform-specific compositing (ultraThinMaterial on iOS, alpha-blended composables on Android)
 - Spotify Web API integration handled natively per platform (iOS SpotifyService actor, Android planned)
 - Apple Music integration planned for iOS only (MusicKit)
 - Gemini AI "Cadence" buddy currently iOS-only, requires tight SwiftUI integration for action cards
-

Screen-by-Screen Breakdown

Feed

- **Purpose:** Social activity feed showing friends' ratings, reviews, and trending albums/songs.
- **iOS:** Screens/FeedScreen.swift (315 lines), FeedViewModel (66 lines). Components: ScreenHeader, SyncBanner, GlassSegmentedControl, SectionHeader, TrendingHeroCard (inline), FeedActivityCard (inline), TrendingSongCard (inline), AlbumArtwork, StarRating, ActionChip, AvatarCircle, SkeletonView. Uses @StateObject + Combine publishers from repo.
- **Android:** ui/screens/FeedScreen.kt (334 lines), FeedViewModel (50 lines). Components: ScreenHeader, SyncBanner, SectionHeader, GlassCard, AlbumArtwork, StarRating, ActionChip, AvatarCircle, EmptyState. Uses collectAsStateWithLifecycle + combine/stateIn.
- **Parity:** Matched. iOS has trending songs toggle (GlassSegmentedControl for Albums/Songs) and featured lists carousel that Android lacks.

Log (Diary)

- **Purpose:** Personal listening journal with quick-rate cards, recent diary entries, and song-level logging.
- **iOS:** Screens/LogScreen.swift (321 lines), LogViewModel (68 lines). Components: ScreenHeader, SyncBanner, GlassSegmentedControl (Albums/Songs toggle), SectionHeader, QuickRateCard (inline), DiaryEntryCard (inline), SongLogEntryCard (inline), TimelineEntry, StarRating, AlbumArtwork, EmptyState, FAB (+), QuickLogSearchSheet (inline). Summary stats with gradient numbers.
- **Android:** ui/screens/LogScreen.kt (352 lines), LogViewModel (47 lines). Components: ScreenHeader, SyncBanner, SectionHeader, GlassCard, AlbumArtwork, StarRating, StatPill, TimelineEntry, FAB (FloatingActionButton), ModalBottomSheet (stub). Summary stats in GlassCard. "Write Later" placeholder card present.
- **Parity:** Mostly matched. iOS has functional QuickLogSearchSheet with Spotify-backed album search; Android sheet is a placeholder. iOS has Songs mode toggle; Android does not.

Search (Discover)

- **Purpose:** Album discovery via search, genre browsing, trending charts.
- **iOS:** `Screens/SearchScreen.swift` (237 lines), `SearchViewModel` (93 lines). Components: `ScreenHeader`, `SyncBanner`, `PillSearchBar`, `SectionHeader`, `TrendingSearchCard` (inline), `GenreCard` (inline), `SearchResultCard` (inline), `TrendChartRow`, `AlbumArtwork`, `StarRating`, `EmptyState`. Spotify live search (local-first, then Spotify merge). Debounced 350ms.
- **Android:** `ui/screens/SearchScreen.kt` (319 lines), `SearchViewModel` (55 lines). Components: `ScreenHeader`, `SyncBanner`, `PillSearchBar`, `SectionHeader`, `GlassCard`, `AlbumArtwork`, `StarRating`, `TrendChartRow`, `EmptyState`. Local search only (no Spotify integration on Android).
- **Parity:** Mostly matched in layout. iOS has live Spotify search merging remote results; Android is local-only.

Lists

- **Purpose:** User-curated album collections (create, view, featured hero).
- **iOS:** `Screens/ListsScreen.swift` (94 lines), `ListsViewModel` (44 lines). Components: `ScreenHeader` (with "Create" action), `SyncBanner`, `SectionHeader`, `FeaturedListHero` (from `ListCards.swift`), `CompactListCard` (from `ListCards.swift`), `EmptyState`, `CreateListSheet` (inline), `PillSearchBar`, `SSButton`.
- **Android:** `ui/screens/ListsScreen.kt` (266 lines), `ListsViewModel` (43 lines). Components: `ScreenHeader` (with "Create" action), `SyncBanner`, `SectionHeader`, `GlassCard`, `AlbumArtwork`, `MosaicCover`, `EmptyState`, `ModalBottomSheet`, `PillSearchBar`, `BlueButton`.
- **Parity:** Matched. Both have create-list sheet, featured hero, compact cards, empty state with CTA.

Profile

- **Purpose:** User profile with avatar, stats, favorites, taste DNA, weekly recap, recent activity.
- **iOS:** `Screens/ProfileScreen.swift` (463 lines), `ProfileViewModel` (145 lines). Components: `SyncBanner`, `AvatarCircle`, `AlbumArtwork`, `StarRating`, `SectionHeader`, `GlassCard`, `StatPill` (inline). Features: hero banner with blurred album art mosaic, animated genre bar chart (`GenreBarRow`), taste stat pills, controversial pick card, Sound DNA summary (Gemini AI-generated 3-word tagline), weekly recap with `ShareLink`, recent activity timeline. Settings gear button navigates to `SettingsScreen`.
- **Android:** `ui/screens/ProfileScreen.kt` (435 lines), `ProfileViewModel` (86 lines). Components: `SyncBanner`, `AvatarCircle`, `GlassCard`, `GlassIconButton`, `AlbumArtwork`, `StatPill`, `SectionHeader`, `EmptyState`, `BlueButton`. Features: profile header card, stat pills row, action buttons (`Share/Export/Settings`), favorite grid, taste DNA tags (horizontal chips), weekly recap card, recent activity list.
- **Parity:** Mostly matched. iOS has animated genre bar chart and AI-generated Sound DNA summary; Android has simpler tag pills. iOS has controversial pick card; Android does not. Both have weekly recap and favorites.

Album Detail

- **Purpose:** Full album view with hero artwork, metadata, rating/review, tracklist, song-level ratings.
- **iOS:** `Screens/AlbumDetailScreen.swift` (454 lines), `AlbumDetailViewModel` (56 lines). Components: `AlbumArtwork`, `StarRating`, `SectionHeader`, `GlassCard`, `SkeletonView`, `AlbumRatingSheet`, `SongRatingSheet`, `ScreenHeader`. Features: hero section, metadata row, segmented tab (`Album/Songs`), rate & review section (opens `AlbumRatingSheet`), tracklist with per-track rating badges, songs breakdown stats, lists containing album, "Also by Artist" section. Navigation: back button + share in toolbar.
- **Android:** No dedicated `AlbumDetailScreen`.
- **Parity:** **iOS-only.** Android has no album detail view.

AI Buddy (Cadence)

- **Purpose:** AI-powered music assistant that can rate albums, draft reviews, search Spotify, and chat.
- **iOS:** Screens/AIBuddyScreen.swift (297 lines), AIBuddyViewModel (210 lines), Services/AIBuddyService.swift (248 lines). Components: CadenceCharacter, CadenceActionCards (478 lines: CadenceReviewCard, CadenceQuickRateCard, CadenceBatchRatingCard, CadenceSearchResultsCard). Uses Gemini 2.5 Flash. Action parsing via regex: [RATE:...], [REVIEW:...], [SEARCH:...]. Suggestion chips, confirmation toasts, thinking animation.
- **Android:** No AI buddy screen or service.
- **Parity:** iOS-only.

Auth

- **Purpose:** Login/signup flow.
- **iOS:** Screens/AuthScreen.swift (180 lines). Components: GlassCard, SSButton, AuthField (inline). Features: toggle between signup/login, email/password/handle fields, error display, 409 conflict fallback (signup -> login). AuthManager handles token persistence in UserDefaults.
- **Android:** No dedicated AuthScreen. Auth is handled silently in RemoteSoundScoreRepository.ensureAuth() using environment variable credentials or hardcoded dev defaults.
- **Parity:** iOS-only UI. Android auto-authenticates without user interaction.

Settings

- **Purpose:** Theme selection, account info, notifications, quiet hours, data export, delete account, sign out.
- **iOS:** Screens/SettingsScreen.swift (378 lines). Components: GlassCard, SSButton, ThemePreviewCard (inline), SettingsRow (inline), ToggleRow (inline). Features: theme carousel (6 themes with live preview), account section, notification toggles (social, recap, comments, reactions), quiet hours stepper, export data (outbox), delete account (API call), sign out, about/version.
- **Android:** No dedicated SettingsScreen. GlassIconButton for "Settings" exists on ProfileScreen but has no navigation target.
- **Parity:** iOS-only.

Splash

- **Purpose:** Animated launch screen with theme cycling, logo reveal, and "Get Started" button.
- **iOS:** Screens/SplashScreen.swift (186 lines). Features: 6-phase animation sequence -- rapid theme cycling (12 cycles), settle on saved theme, logo spring scale, title slide-up, subtitle fade, button appear. Uses DispatchQueue timers.
- **Android:** No splash screen.
- **Parity:** iOS-only.

Component Libraries

iOS (27 components)

Name	File	Used By	Categ
ActionChip	Components/ActionChip.swift	FeedScreen	Interac

AlbumArtwork	Components/AlbumArtwork.swift	Feed, Log, Search, Profile, AlbumDetail	Media
AlbumRatingSheet	Components/AlbumRatingSheet.swift	AlbumDetailScreen	Sheet
AppBackdrop	Components/AppBackdrop.swift	ContentView, AIBuddy, Settings, AlbumDetail	Layout
AvatarCircle	Components/AvatarCircle.swift	FeedScreen, ProfileScreen	Display
CadenceActionCards	Components/CadenceActionCards.swift	AIBuddyScreen	AI/Char
CadenceCharacter	Components/CadenceCharacter.swift	AIBuddyScreen	AI/Char
EmptyState	Components/EmptyState.swift	Feed, Log, Search, Lists	Feedba
FloatingTabBar	Components/FloatingTabBar.swift	ContentView	Naviga
GlassCard	Components/GlassCard.swift	All screens	Layout
GlassIconButton	Components/GlassIconButton.swift	ProfileScreen	Interac
GlassSegmentedControl	Components/GlassSegmentedControl.swift	FeedScreen, LogScreen	Interac
ListCards	Components/ListCards.swift	ListsScreen, FeedScreen	Display
MosaicCover	Components/MosaicCover.swift	ListsScreen	Media
PillSearchBar	Components/PillSearchBar.swift	SearchScreen, LogScreen, ListsScreen	Input
ReviewSheet	Components/ReviewSheet.swift	AlbumDetailScreen	Sheet
ScreenHeader	Components/ScreenHeader.swift	All screens	Layout
SectionHeader	Components/SectionHeader.swift	All screens	Layout
SkeletonView	Components/SkeletonView.swift	FeedScreen, AlbumDetailScreen	Feedba
SongRatingSheet	Components/SongRatingSheet.swift	AlbumDetailScreen	Sheet
SSButton	Components/SSButton.swift	AuthScreen, ListsScreen, Settings	Interac
StarRating	Components/StarRating.swift	Feed, Log, Search, Profile, AlbumDetail	Interac

StatPill	Components/StatPill.swift	ProfileScreen	Display
SyncBanner	Components/SyncBanner.swift	All screens	Feedba
Tab	Components/Tab.swift	FloatingTabBar	Naviga
TimelineEntry	Components/TimelineEntry.swift	LogScreen	Layout
TrendChartRow	Components/TrendChartRow.swift	SearchScreen	Display

Android (7 component files, ~20 composables)

Name	File	Used By	Category	Li
AlbumArtPlaceholder	components/AlbumArtPlaceholder.kt	All screens (via AlbumArtwork)	Media	10
AppBackdrop	components/AppBackdrop.kt	SoundScoreApp	Layout	41
GlassCard	components/GlassCard.kt	All screens	Layout	10
NewComponents	components/NewComponents.kt	All screens (AvatarCircle, PillSearchBar, SyncBanner, EmptyState, TimelineEntry, GlassIconButton)	Mixed	32
PremiumComponents	components/PremiumComponents.kt	Screens (ScreenHeader, SectionHeader, ActionChip, StatPill, TrendChartRow, BlueButton, MosaicCover)	Mixed	3C
SoundScoreButton	components/SoundScoreButton.kt	Screens (BlueButton alias)	Interaction	72
StarRating	components/StarRating.kt	Feed, Log, Search, Profile	Interaction	8C

Data Flow

ios

```
SeedData (hardcoded) → SoundScoreRepository (singleton, @Published properties)
→ ViewModel (ObservableObject, Combine publishers via $property.assign(to:))
→ Screen (@StateObject, SwiftUI reactivity)
```

- Repository is a singleton (`SoundScoreRepository.shared`) with `@Published` properties.
- ViewModels subscribe using Combine's `$property.receive(on: RunLoop.main).assign(to:)` pattern.
- Multi-stream merging via `Publishers.CombineLatest` , `CombineLatest3` .
- Screens use `@StateObject` for ViewModel ownership.
- `@MainActor` isolation on `AlBuddyViewModel`. Other ViewModels dispatch to `MainActor` via `.receive(on: RunLoop.main)` .

Android

```
SeedData (DummyData.kt) → RemoteSoundScoreRepository (MutableStateFlow properties)
  → ViewModel (combine/stateIn → StateFlow<UiState>)
  → Screen (collectAsStateWithLifecycle)
```

- Repository implements `SoundScoreRepository` interface, exposed via `AppContainer.repository` singleton.
- Uses `MutableStateFlow` for all state. Updates via `.update {}` (immutable copy) or `.value =` .
- ViewModels use `combine()` to merge multiple flows into a single `UiState` data class.
- `stateIn(scope = viewModelScope, started = WhileSubscribed(5_000))` for lifecycle-aware sharing.
- Screens call `collectAsStateWithLifecycle()` for Compose-aware collection.

Offline Architecture

ios

```
User action → optimistic local state update
  → OutboxStore.enqueue(OutboxOperation)
  → SoundScoreRepository.syncOutbox()
  → OutboxSyncEngine.flush(handler:)
  → API call per operation
  → markDispatched (success) or markFailed (exponential backoff)
```

- `InMemoryOutboxStore` : `@Published` var `pending`: `[OutboxOperation]`
- `OutboxSyncEngine` : iterates pending ops, calls handler closure, marks dispatched/failed.
- Backoff: `pow(2, min(attemptCount, 6))` seconds.
- `SyncBanner` component shows "Pending N offline ops" when outbox is non-empty.

Android

```
User action → optimistic local state update (MutableStateFlow.update)
  → InMemoryOutboxStore.enqueue(OutboxOperation)
  → RemoteSoundScoreRepository.syncOutbox()
  → OutboxSyncEngine.flush(handler:)
  → API call per operation (Retrofit)
  → markDispatched (success) or markFailed (backoff)
```

- `InMemoryOutboxStore` : `StateFlow<List<OutboxOperation>>` via `MutableStateFlow` .
- `OutboxSyncEngine` : same flush pattern, marks dispatched/failed.

- Backoff: `2^min(count,6) * 1000` milliseconds.
- Sync banner wired to `syncMessage` `StateFlow`.

Operation Types

iOS	Android	Description
<code>rateAlbum</code>	<code>RATE_ALBUM</code>	Rate an album (0-6 scale)
<code>rateTrack</code>	--	Rate individual track (iOS only)
<code>createReview</code>	--	Save album review (iOS only)
<code>toggleReaction</code>	<code>TOGGLE_REACTION</code>	Like/unlike feed item
<code>createList</code>	<code>CREATE_LIST</code>	Create new user list
<code>exportData</code>	<code>EXPORT_DATA</code>	Export user data snapshot
<code>registerDeviceToken</code>	<code>REGISTER_DEVICE_TOKEN</code>	Push notification token
<code>updateNotificationPreferences</code>	<code>UPSERT_NOTIFICATION_PREFERENCES</code>	Notification settings
--	<code>GENERATE_RECAP</code>	Trigger recap generation (Android only)

Theme System

iOS

- **ThemeManager** (`ObservableObject` , singleton): stores current `AccentTheme` in `UserDefaults` (`ss_accentTheme`).
- **6 themes:** Emerald, Bonfire, Rose, Amethyst, Midnight, Gilt.
- Each theme defines: `primary` , `secondary` , `primaryDim` , `secondaryDim` , `backdropGlow` , `backdropSecondaryGlow` , and a `ThemeColorScheme` (`darkBase`, `darkSurface`, `darkElevated`).
- **SSColors** enum: theme-adaptive backgrounds (`darkBase` , `darkSurface` , `darkElevated` delegate to `ThemeManager`), plus fixed glass, chrome, accent, overlay, and text colors.
- **SSTypography** enum: 12 static font styles, all `.rounded` design, sizes 10-28pt.
- **Glassmorphism:** `GlassCard` uses `ultraThinMaterial` + white-alpha overlay + border stroke. `AppBackdrop` provides `RadialGradient` glows from theme primary/secondary.

Android

- **Material3 dark color scheme** (`darkColorScheme`): `primary=AccentGreen`, `secondary=AccentAmber`, `tertiary=AccentCoral`.
- **Single theme only** (no theme switcher). Hardcoded in `SoundScoreTheme` composable.
- **Color.kt:** 79 lines. `DarkBase`, `DarkSurface`, `DarkElevated`, Glass variants, Chrome variants, Accent colors, `AlbumColors` object with 10 gradient pairs.
- **Type.kt:** 108 lines. Custom `SoundScoreTypography` using Inter/system fonts.

- **Glassmorphism:** `GlassCard` composable with alpha-blended backgrounds + border strokes. `AppBackdrop` uses `Box` with `RadialGradient`.
 - **No per-theme dark base colors.** Android uses fixed dark palette.
-

Third-Party Integrations

Spotify Web API (iOS only)

- **SpotifyService** (`actor` , `singleton`): thread-safe with Swift actor isolation.
- **Auth:** Client credentials flow (`client_id:client_secret` base64 -> `/api/token`).
- **Endpoints used:**
 - `GET /v1/search?type=album` -- album search with artwork, artist, year, genres.
 - `GET /v1/albums/{id}` -- album detail (richer genre data).
 - `GET /v1/albums/{id}/tracks` -- full tracklist with durations.
- **Caching:** In-memory `artworkCache: [String: String]` keyed by `"title|artist"` .
- **Rate limiting:** 150ms sleep between sequential artwork enrichment calls.
- **Token management:** Cached token with 60-second safety margin before expiry.
- **Used by:** `SearchViewModel` (live search merge), `SoundScoreRepository` (artwork enrichment on init, track fetching), `AlBuddyViewModel` (search action handling).
- **Android:** No Spotify integration. Album artwork relies on seed data and API-provided URLs only.

Gemini AI — Cadence (iOS only)

- **AlBuddyService** (`actor` , `singleton`): calls Gemini 2.5 Flash via REST.
 - **Model:** `gemini-2.5-flash` at `generativelanguage.googleapis.com/v1beta` .
 - **System prompt:** Rich personality definition ("dorky, passionate music nerd"), action tag instructions, full album catalog injection, user listening context.
 - **Action parsing:** Regex-based extraction from response text:
 - `[RATE:album_id:Album Title:rating]` -- suggest rating
 - `[REVIEW:album_id:Album Title:review text]` -- draft review
 - `[SEARCH:query]` -- trigger Spotify search
 - **Generation config:** temperature 0.9, `maxOutputTokens` 1000.
 - **Sound DNA summary:** Separate Gemini call in `ProfileViewModel` (temperature 1.0, `maxOutputTokens` 20) to generate 3-word taste tagline. Cached in `UserDefaults`.
 - **Android:** No AI integration.
-

Auth Flow

iOS

- **AuthManager** (`ObservableObject` , `singleton`):
 - `@Published isAuthenticated: Bool` , `@Published currentHandle: String?`
 - Token storage: `UserDefaults` keys `ss_accessToken` , `ss_refreshToken` , `ss_handle` .
 - Endpoints: `POST /v1/auth/login` , `POST /v1/auth/signup` , `POST /v1/auth/refresh` .
 - `devAutoSignup()` in `#if DEBUG` : attempts signup with hardcoded dev credentials, falls back to login on 409 conflict.
 - Auto-refresh: not yet wired to 401 interceptor (manual refresh method exists).

- **Flow:** SplashScreen -> ContentView checks `authManager.isAuthenticated` -> AuthScreen (login/signup) or main app.
- **AuthScreen:** Toggle between signup/login, validates fields, calls `authManager.signup/login`, triggers `SoundScoreRepository.shared.refresh()` on success.

Android

- **No AuthManager class.** Auth handled inside `RemoteSoundScoreRepository.ensureAuth()`.
 - **ensureAuth():** Reads credentials from environment variables (`SOUNDCORE_DEV_EMAIL`, etc.) with hardcoded fallbacks. Tries login first, falls back to signup.
 - **Token storage:** In-memory `accessToken: String?` only. No persistence across app restarts.
 - **No AuthScreen.** Auto-authenticates silently on repository init.
 - **No token refresh.** If token expires, next API call fails until app restart.
-

What Works

1. **Glassmorphism design system** is visually consistent across both platforms. GlassCard, glass borders, dark elevated surfaces, and chrome text hierarchy create a cohesive look.
 2. **Offline-first outbox architecture** is fully implemented on both platforms with idempotency keys, exponential backoff, and optimistic UI updates.
 3. **Feed/Log/Search/Lists/Profile** core screens are present and functional on both platforms with matching layouts and data flows.
 4. **Spotify artwork enrichment** (iOS) provides real album art on launch, significantly improving visual quality over gradient placeholders.
 5. **AI Buddy (Cadence)** on iOS is a genuinely useful feature -- it can rate albums, draft reviews in the user's voice, and search Spotify, all with rich action cards.
 6. **Theme system** (iOS) with 6 selectable themes and per-theme dark palettes gives users real personalization.
 7. **Repository pattern** is cleanly separated on both platforms -- ViewModels never touch API/network directly.
 8. **Combine pipeline** (iOS) and **Flow combine/stateIn** (Android) provide reactive, lifecycle-aware data flow.
 9. **Deep link resolver** (Android) with `DeepLinkResolver.kt` and tests provides foundation for universal links.
 10. **SeedData** on both platforms allows the app to function offline with realistic mock data immediately on launch.
-

What's Broken

1. **Android is missing 4 screens:** AlbumDetailScreen, AIBuddyScreen, AuthScreen, SettingsScreen, SplashScreen. These are all functional on iOS.
2. **Android has no Spotify integration.** No artwork enrichment, no live search, no tracklist fetching.
3. **Android has no AI/Gemini integration.** No Cadence chat, no review drafting, no AI-assisted rating.
4. **Android auth has no persistence.** Token is in-memory only; lost on every app restart. No refresh token flow.
5. **Android has no theme switcher.** Single hardcoded Emerald theme vs. iOS's 6 selectable themes.
6. **iOS test coverage is zero.** No unit tests, no UI tests, no snapshot tests. Android has 4 test files (`DeepLinkResolverTest`, `OutboxSyncEngineTest`, `ScreenSmokeTest`, `SoundScoreRepositoryMappingTest`, `ScreenPresentationTest`).
7. **No strings.xml / Localizable.strings.** All user-facing text is hardcoded inline on both platforms. No i18n support.

8. **iOS SoundScoreRepository mutates state in-place** (e.g., `feedItems[index].isLiked.toggle()`, `albums[index].artworkUrl = url`). This violates immutability principles and can cause subtle SwiftUI update bugs.
 9. **iOS Secrets.swift** likely contains hardcoded API keys (Spotify clientId/clientSecret, Gemini API key). These should be in build config or keychain.
 10. **Android components are consolidated into 2 mega-files** (NewComponents.kt at 320 lines, PremiumComponents.kt at 301 lines) rather than one-component-per-file. This hurts discoverability and violates the codebase's own file organization conventions.
 11. **No error retry UI on Android.** iOS has `ErrorBanner` with retry button; Android sync errors only show in `SyncBanner`.
 12. **iOS Log/Feed screens have trending songs toggle** (`GlassSegmentedControl`) that Android lacks.
 13. **No push notification implementation.** Both platforms have outbox ops for device token registration but no actual FCM/APNs integration.
 14. **Weekly recap "Generate" is one-way.** Android can trigger generation via outbox, but there is no UI to initiate it from a button press.
-

Actionable Next Steps

P0 — Critical (blocks release)

1. **Add Android AlbumDetailScreen.** Without it, users cannot view album details, rate albums, or see tracklists. Port from iOS's 454-line implementation.
2. **Add Android AuthScreen.** Silent dev-auth is not shippable. Port the iOS AuthScreen (180 lines) to Compose.
3. **Persist Android auth tokens.** Move from in-memory to EncryptedSharedPreferences. Add token refresh on 401.
4. **Move iOS secrets out of source.** Migrate Spotify/Gemini keys to xcconfig or build environment injection.

P1 — High (needed for quality parity)

5. **Add Android SettingsScreen.** Port theme selection (with Material3 dynamic color or manual theme switching), notification toggles, export, delete account, sign out.
6. **Add Android SplashScreen.** Port the 6-phase animation or use Android 12+ SplashScreen API with a simpler fallback.
7. **Add Spotify integration to Android.** Create a SpotifyService class (Retrofit-based) mirroring the iOS actor. Wire into SearchViewModel and artwork enrichment.
8. **Split Android mega-component files.** Break NewComponents.kt and PremiumComponents.kt into individual files (AvatarCircle.kt, PillSearchBar.kt, SyncBanner.kt, etc.).
9. **iOS test coverage.** Add unit tests for ViewModels (FeedViewModel, LogViewModel, SearchViewModel) and OutboxStore. Target 80% on business logic.
10. **Extract all hardcoded strings.** Create strings.xml (Android) and Localizable.strings (iOS) with all user-facing copy.

P2 — Medium (polish and features)

11. **Add AI Buddy to Android.** Port AIBuddyService, AIBuddyViewModel, AIBuddyScreen, and CadenceActionCards to Compose.
12. **Add Android theme switcher.** Port the 6-theme system (Emerald, Bonfire, Rose, Amethyst, Midnight, Gilt) with per-theme dark palettes.
13. **Fix iOS state mutation.** Refactor SoundScoreRepository to use immutable copy-on-write patterns instead of in-place array mutation.

14. **Add trending songs toggle to Android Feed.** Port GlassSegmentedControl and TrendingSongCard.
15. **Add error retry UI to Android.** Port iOS ErrorBanner component with retry callback.
16. **Wire push notifications.** Integrate FCM (Android) and APNs (iOS) with the existing device token registration outbox ops.

P3 — Low (nice to have)

17. **Add Android deep link handling end-to-end.** DeepLinkResolver exists with tests but is not wired to navigation.
18. **Add iOS snapshot tests.** Capture GlassCard, StarRating, AlbumArtwork renders.
19. **Add rate-limiting/caching to Android API calls.** iOS has Spotify token caching and artwork cache; Android has neither.
20. **Add album art color extraction.** Both platforms use hardcoded AlbumColors. Use Palette (Android) or dominant-color extraction (iOS) from fetched artwork.